

Developing Software as a Product: Optimizing Butterfly Image Classifier Prototype

Group 2: Kai Aebli, Pia Lagler, Mariia Gurova

1 Goal of the project

The goal of the project is to optimize an old project ([Project Link](#)) in according to the best practices in software engineering. The project was created in the first years of studying in data science program, therefore the focus back then was on the reaching the best prediction accuracy and not on the code structure, reproducibility and maintainability. The project delivers the butterfly classification and classes prediction on images.

For this project the following goals to improve the code quality are defined:

- Improve the existing codebase into a maintainable repository.
- Cleanup the existing scripts according to best practice coding guidelines.
- Implement modular structure with classes definition in isolated scripts instead of a single file in .ipynb.
- Implement testing and CI/CD pipelines.
- Expand the README for better instructions.

2 Language/library choices

The project is implemented in python, as it is the primary programming language used throughout ACLS courses and the students are most familiar with it. In addition, the neural network was built using Keras, a Python-based library that provides a simple interface for building neural networks (Rao, 2023). Therefore, the code is kept in Python and the focus is put on improving its structure.

3 Project workflow setup

Initially, the project had an unclear and simplified structure. Following problems were detected:

- No dedicated folders with meaningful names
- The training script is fully presented in .ipynb and quite long
- README.md does not contain the exact explanation of scripts
- Training results and models are saved in the main directory
- Data folder is called “leedsbutterfly” which is not intuitive and requires time to realize that it is a data folder
- Images are loaded directly in the repository and take more space
- Only main branch, no reviews or approvals from another collaborator implemented
- No workflows and actions implemented

Overall, if another developer would fork or pull the repository, it would require long time to understand the code architecture, since the main script lacked modularity and encapsulation.

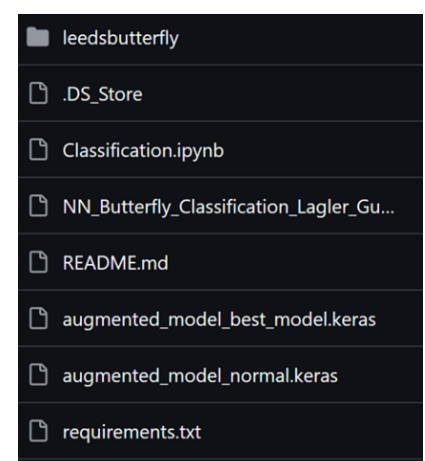


Figure 1: Initial directory structure of the project

3.1 Refactoring

Firstly, during refactoring, some dead code lines and code redundancy was detected, therefore the first task was to clean it.

Secondly, new structure was chosen. The original code was only implemented in a single script. Therefore, at first the code was refactored into modular Python files. The *main.py* was created in the root repository with referring to the isolated scripts for specific functions (including data loading, training, visualization) in *scripts/* directory. That allowed easier readability and code understanding for any further user.

Thirdly, the configuration was implemented (see [config.py](#)). With that approach, instead of changing parameters for training such as number of epochs, batch size, learning rates in the main script, developers can define them in configuration file directly without searching for these values in the main or separate scripts.

Additionally, the dataset with images was removed from the repository. In the improved version a script for data download was added to speed up the cloning process. Now the images first will be downloaded to the *data/* folder, and the training will start only after the data download.

Moreover, the functions names were corrected or renamed for direct clear interpretation. If earlier almost all the functions contained comments, now due to the clear naming that was not necessary. The code was rewritten with object-oriented principles with classes and methods for separate operations (such as model definition, training, result visualization).

Furthermore, initially the imported libraries were stacked on each other at the beginning of the script. With the isolated scripts, the number of imports was reduced, resulting in a better overview.

Dependencies import

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import os
import random
from sklearn.model_selection import train_test_split
from tensorflow.keras.preprocessing.image import load_img, img_to_array, ImageDataGenerator
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout, BatchNormalization
from tensorflow.keras.metrics import Precision, Recall, BinaryAccuracy, SparseCategoricalAccuracy, Precision
from tensorflow.keras.models import load_model
import seaborn as sns
import math
from tensorflow.keras.callbacks import TensorBoard, EarlyStopping, ReduceLROnPlateau
from keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.optimizers import Adam
from sklearn.metrics import confusion_matrix
from collections import Counter
```

Figure 3: Imports at the project initial state

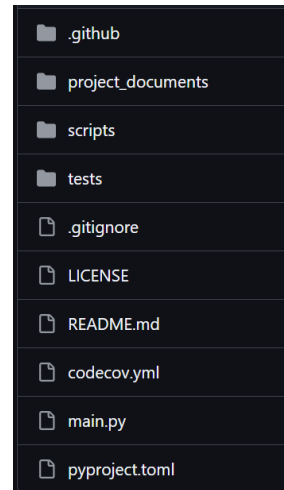


Figure 2: Refined Project Structure

3.2 Branches

Originally, the project was committed only on the main branch. For this project, main branch was protected from direct pushes and changes were merged through pull requests to keep the main branch in a functional state. All the work was shifted to a *wip* (work in project) branch. For the separate bugs, fixes team members created separate branches which were merged firstly to *wip*. Moreover, collaborators implemented more frequent pushes with smaller changes to have a better overview of all the changes in GitHub.

However, during the project, several mistakes were made. For example, after creating the *wip* branch, the first commits were still accidentally pushed to *main* branch due to habits and had to be pushed back and moved to *wip*. Also, deleting the data folder became inconvenient after the branch protection was enabled, as the folder contained more than 1,000 files. Ideally, the data should have been removed before protecting the branch. In addition, multiple features were implemented in the same branch because team was not used to creating a new branch in between for small incremental changes. Each of these branches was thereafter merged into *wip* using a PR and deleted (see Section 3.3).

3.3 PRs

During the project, a pull request for each merge to *wip* branch was configured. Each PR was reviewed by one of the teammates as well as Copilot. Besides that, the team tried to link the related issued in each PR documentation, however sometimes it was forgotten and the issues had to be closed manually.

3.4 CI/CD

To automate code quality assurance, a CI/CD pipelines with GitHub Actions were implemented (see [workflows](#)). To ensure that the same settings are used locally and in the CI environment, the configuration is defined in *pyproject.toml* file.

The pipelines were set to run automatically on every push to the main branch and for each PR to avoid formatting issues before merging. In total there are 3 workflows:

1. *ci.yml*:
Runs the main quality checks. It installs the project, checks code style with *Ruff* and *Black*, runs the test suite with *pytest* and measures coverage. The pipeline requires to have at least 80% coverage. The generated *coverage.xml* file is uploaded to Codecov, where coverage changes can be tracked and shown directly in pull requests.
2. *matrix.yml*:
The workflow also runs on pushes to main and pull requests. It tests the project on Python 3.10, 3.11, and 3.12 to make sure the code works across supported Python versions. It also runs *Pylint*, which must reach a minimum score of 8.0/10.
3. *build_binaries.yml*:
Builds standalone executable files for Linux, Windows and macOS using *PyInstaller* and used for releases. It runs when a version tag (e.g. v1.0.0) is pushed or when started manually. The created binaries were uploaded to GitHub Release.

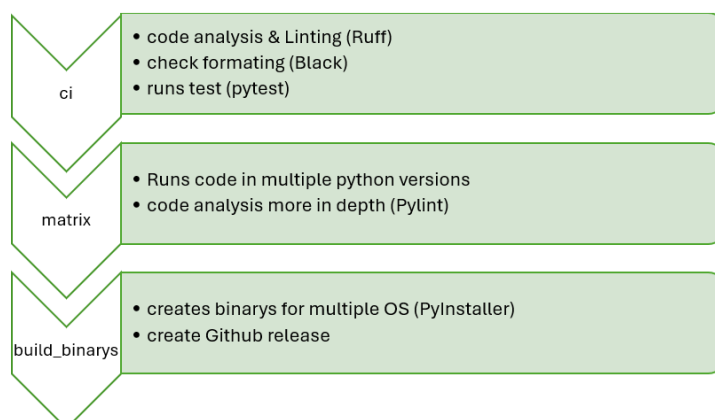


Figure 4: visual representation of workflows

Dependabot is also configured to keep dependencies up to date. It creates weekly pull requests for outdated Python dependencies and GitHub Actions versions. It runs automatically over Github.

5 Appendix

```
# Load the data
folder_path = "../leedsbutterfly/images"

# Read all filenames in the folder
filenames = os.listdir(folder_path)

# Extract categories from the filenames
categories = {}
for filename in filenames:
    imagename, _ = os.path.splitext(filename)
    if imagename[1:3].isdigit():
        # Extract the category
        category = int(imagename[1:3])
        categories[filename] = category

# Preprocessing images
image_size = (128, 128)
images = []
original_labels = []
for filename in categories.keys():
    image_path = os.path.join(folder_path, filename)
    img = load_img(image_path, target_size=image_size)
    img_array = img_to_array(img)
    images.append(img_array)

# Set original_labels based on categories
original_labels.append(categories[filename])

print(f"There are total {len(images)} images in this dataset.")
```

Figure 7: Main script structure at the beginning

Name	Stmts	Miss	Cover	Missing
scripts\config.py	42	7	83%	15-24
scripts\data_download.py	46	0	100%	
scripts\data_preprocessing.py	82	0	100%	
scripts\evaluator.py	27	0	100%	
scripts\model.py	30	0	100%	
scripts\train.py	18	0	100%	
scripts\visualizer.py	130	0	100%	
TOTAL	375	7	98%	
Required test coverage of 80% reached. Total coverage: 98.13%				

Figure 6: Test Coverage Results

6 Bibliography

- Gregg, B. (2026). *Brendangregg/FlameGraph* [Perl]. <https://github.com/brendangregg/FlameGraph> (Original work published 2011)
- microsoft/debugpy: An implementation of the Debug Adapter Protocol for Python*. (n.d.). Retrieved 7 May 2026, from <https://github.com/microsoft/debugpy>
- Okan Yenigün. (2025, November 19). *Ruff: The Fastest Python Linter and Formatter You'll Ever Use*. <https://python.plainenglish.io/ruff-the-fastest-python-linter-and-formatter-youll-ever-use-a311b00a195f>
- Ph.D, M. R. (2025, August 14). Ruff Tutorial: A Complete Guide for Python Developers. *Medium*. <https://medium.com/@amjadraza24/ruff-tutorial-a-complete-guide-for-python-developers-1aa62272596d>
- Pytest documentation*. (n.d.). Retrieved 7 May 2026, from <https://docs.pytest.org/en/stable/>
- Python debugging in VS Code*. (n.d.). Retrieved 7 May 2026, from <https://code.visualstudio.com/docs/python/debugging>
- Rao, M. N. (2023). A Comparative Analysis of Deep Learning Frameworks and Libraries. *International Journal of Intelligent Systems and Applications in Engineering*, 11(2s), 337–342.
- Zoo Codes. (2023, July 13). *Enhancing Python Code Quality: A Comprehensive Guide to Linting with Ruff*. DEV Community. https://dev.to/ken_mwaura1/enhancing-python-code-quality-a-comprehensive-guide-to-linting-with-ruff-3d6g

AI Tools

ChatGPT (OpenAI, 2026) and Microsoft Copilot (Microsoft, 2026) were used for the purposes of coding, troubleshooting and debugging.